When Policies Hesitate: Uncertainty-Aware Credit Assignment for Flow-Based Robot Policy Optimization

Anonymous CVPR submission

Abstract

Recent vision-language-action (VLA) models and world-action models (WAMs) use flow matching to model complex continuous action distributions. Online post-training can further improve these policies, but typically relies on trajectory-level feedback that overlooks substantial differences in the utility of individual actions for policy improvement, limiting sample efficiency and performance gains. Process reward models and critics provide finer-grained feedback at the cost of additional training. We propose Uncertainty-Aware Credit Assignment (UACA), which estimates action entropy from actions generated with different initial noise samples under the same observation to guide action-level credit assignment. We find that high uncertainty is concentrated in a small subset of actions associated with decision-sensitive stages of task execution, and that these actions offer greater potential for improvement than low-uncertainty actions the policy has already mastered. This observation motivates reweighting advantages or losses to prioritize learning from high-uncertainty actions. UACA requires no additional training, integrates as a plug-and-play component into existing post-training methods (e.g., EXPO-FT, RL Token, and GRPO) while preserving their underlying policy optimization objectives, and applies broadly to flow-based policies (e.g., $\pi_{0.5}$ and OpenWAM). Simulation and real-world experiments demonstrate consistent improvements in sample efficiency or task success rate, supporting the use of policy uncertainty to guide online post-training of flow-based policies.

1. Introduction

Vision-language-action (VLA) models [3, 4] and world-action models (WAMs) [2, 6, 19, 39, 42] based on flow matching have proven effective for robotic manipulation. Through behavior cloning on large-scale offline expert demonstrations, these models capture complex continuous action distributions and acquire diverse manipulation skills. Collecting expert demonstrations is time-consuming and costly, and policies obtained through supervised fine-tuning may still fall short of desired performance, such as precision-demanding tasks.

Online post-training offers a natural way to further improve these policies using experience collected through interaction. Some recent methods focus on stable policy improvement in parallel simulation [7, 21], while others prioritize real-world sample efficiency [10, 16, 25, 37, 40]. These methods typically use trajectory-level feedback to optimize policies, assigning a shared advantage to all actions or propagating feedback through temporal-difference learning. However, actions within a trajectory differ in their impact on task outcomes and may also differ in their value for policy improvement. Failing to identify and prioritize these actions limits sample efficiency and performance gains.

Recent efforts have primarily relied on training auxiliary models, including process reward models [8, 13, 18, 23, 36] and critics [12, 26]. Although effective, these approaches incur additional training costs and may also require elaborate, hand-designed mechanisms. Can we identify actions with greater value for policy improvement without additional training, and use this information to guide diverse online post-training methods in a simple, plug-and-play manner?

Policy uncertainty during action generation may help identify such actions, as higher uncertainty can indicate greater room for improvement. We quantify policy uncertainty using action entropy estimated from multiple action samples conditioned on the same observation. We find that high uncertainty is concentrated in a small subset of actions, typically associated with decision-sensitive stages of task execution, and that these actions offer greater potential for improvement than low-uncertainty actions the policy has already mastered. For example, further optimizing free-space motion that the policy has already mastered may yield smaller gains than correcting alignment errors during precise insertion.

Motivated by this finding, we propose Uncertainty-Aware Credit Assignment (UACA), which uses action entropy to adjust the relative contribution of each action within a trajectory to policy updates. Depending on the underlying post-training method, UACA simply reweights training signals, such as advantages or losses, to prioritize high-

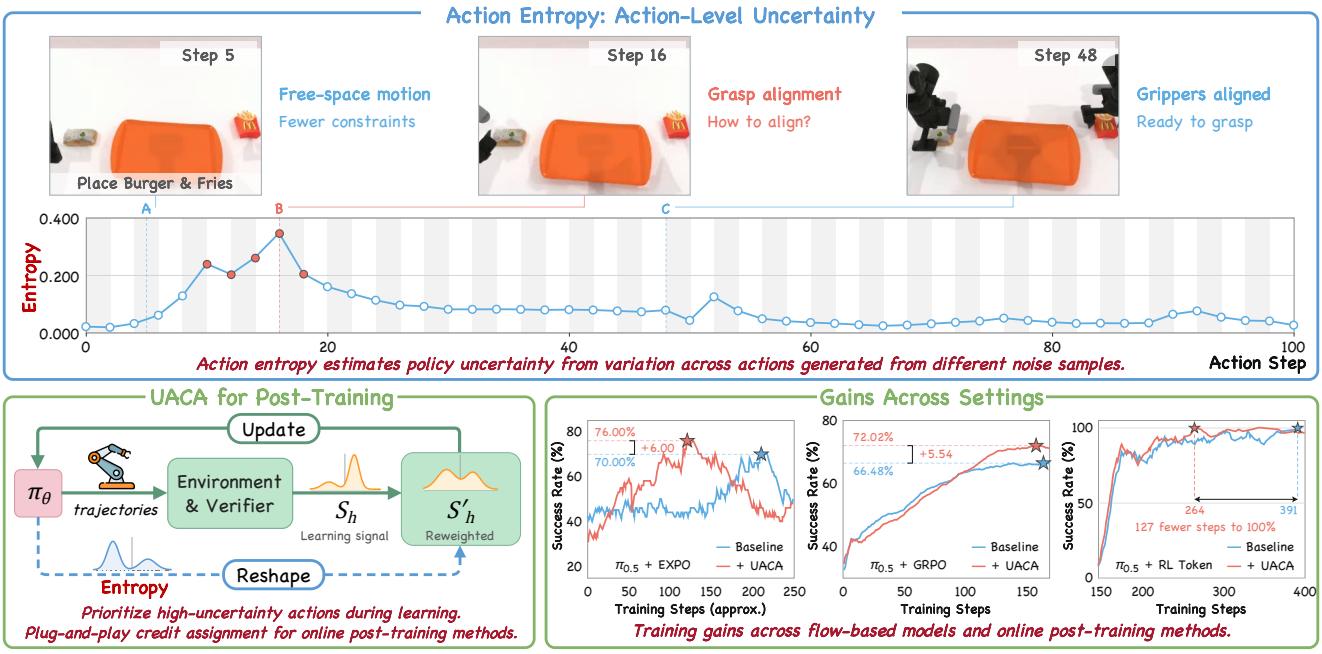


Figure 1. **Overview of UACA.** UACA estimates action entropy from actions generated with different initial noise samples under the same observation, and uses it to reweight advantages or supervised losses during online post-training, prioritizing high-uncertainty actions. This mechanism integrates with different flow-based robot policies and post-training algorithms, improving success rates and sample efficiency in simulation and real-world tasks.

uncertainty actions. UACA can thus be plugged into online post-training methods such as EXPO-FT, RL Token, and GRPO, and applied to flow-based policies such as $\pi_{0.5}$ and OpenWAM, without requiring additional training of auxiliary models. Our approach is based on a simple intuition: actions with higher policy uncertainty should receive greater emphasis during training.

To sum up, our contributions are:

- Using action entropy to quantify policy uncertainty, we find that high-uncertainty actions are often concentrated in a few critical stages and offer greater potential for policy improvement.
- We propose Uncertainty-Aware Credit Assignment (UACA), which upweights high-uncertainty actions during training and supports plug-and-play integration into multiple online post-training methods without additional training of auxiliary models.
- Extensive experiments across simulated and real-world environments, flow-based robot policies, and online post-training methods demonstrate that UACA consistently improves sample efficiency or task success rates.

2. Related Work

2.1. Generative Robot Policies

Generative robot policies learn conditional action distributions and generate actions through iterative denoising [9, 43]. Building on vision-language pretraining, flow-based vision-language-action (VLA) models support language-conditioned control and cross-task generalization [3, 4, 44]. World-action models (WAMs) further combine video prediction with action learning to incorporate environmental dynamics into action generation [2, 5, 6, 17, 19, 22, 39, 41, 42]. UACA targets these flow-based policies and uses intrinsic policy uncertainty to guide their online post-training.

2.2. Online Post-Training of Robot Policies

Online post-training uses interaction experience to improve policies initialized by supervised fine-tuning (SFT). One line of work uses policy gradients and group-relative optimization to optimize task returns in parallel simulation [7, 21]. Another uses off-policy actor–critic methods to reuse experience and builds on pretrained policies for sample-efficient adaptation [1, 10, 25, 27, 37, 40]. Online methods based on filtered imitation learning iteratively collect and select experience for supervised policy updates [24]. Systematic studies of batch online RL further examine how algorithm choice and policy extraction affect learning from autonomously collected data [11]. These methods provide different optimization interfaces, within which UACA adjusts the relative training weights of individual actions.

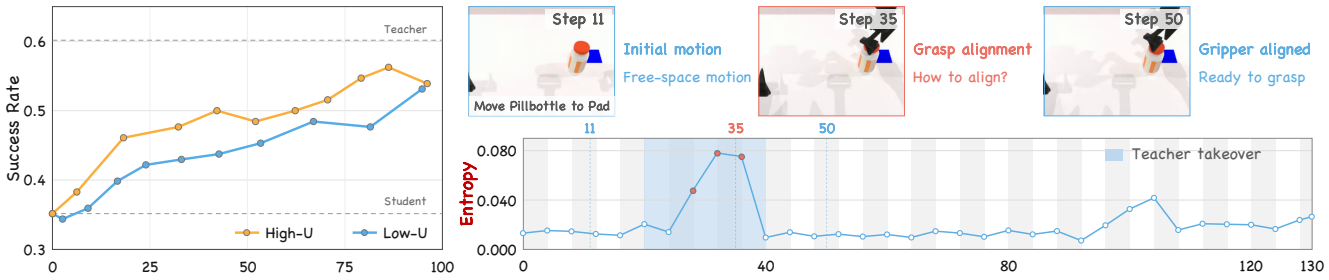


Figure 2. **Left:** Success rate versus the fraction of the first three action chunks taken over by a teacher policy fine-tuned with GRPO, comparing preferential takeover of high- and low-uncertainty chunks. High-uncertainty actions offer greater potential for improvement. **Right:** An example trajectory in which the teacher takes over the high-entropy second action chunk.

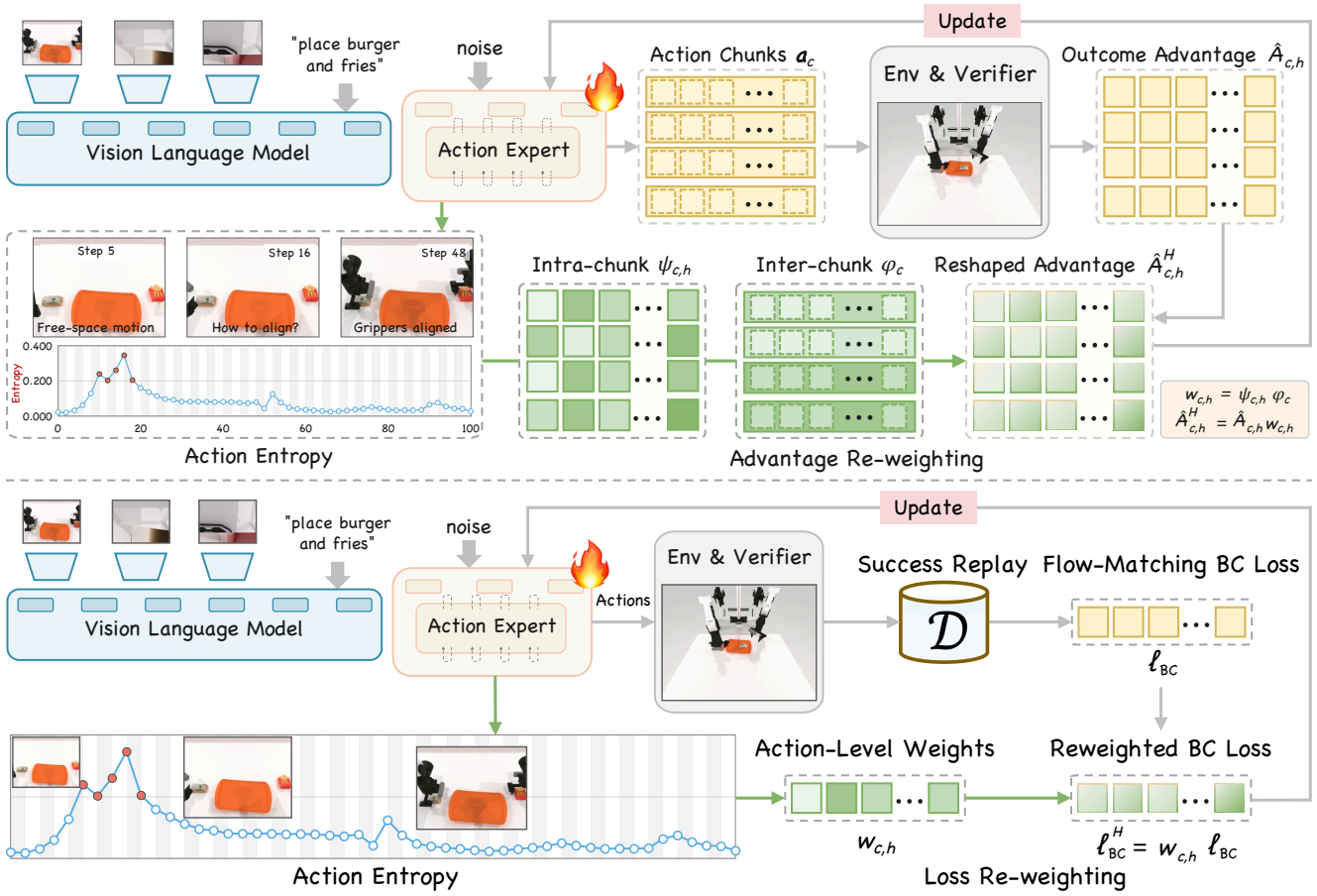


Figure 3. UACA estimates action entropy from actions generated with different initial noise samples at inference time and prioritizes learning from high-uncertainty actions. It applies a common reweighting principle across post-training methods, illustrated here with GRPO (top) and filtered behavior cloning (Filtered BC, bottom).

2.3. Fine-Grained Credit Assignment and Intrinsic Policy Signals

Process reward and value models derive local learning signals from task progress [8, 18]; some methods weight policy updates using estimated advantages [12, 14, 26], while others use heuristic reweighting [28]. In language models, token entropy is often used to adjust token-level advantages [20, 38]. Similarly, internal denoising dynamics in flow-based robot policies provide uncertainty signals for downstream applications [15, 29–31, 35]. UACA connects intrinsic signals from flow-based policies to fine-grained credit assignment by using action entropy to reweight advantages or supervised losses during online post-training.

3. Method

UACA uses intrinsic action uncertainty to adjust the relative contribution of individual actions to policy updates during online post-training. Given the same observation, we generate action samples from multiple initial noise samples and quantify policy uncertainty using action entropy. We then convert action entropy into training weights to focus learning on actions with greater potential for policy improvement. We apply this reweighting mechanism to advantages, actor objectives, and supervised losses across online post-training methods.

3.1. Policy Uncertainty from Action Entropy

Flow-based robot policies map initial noise to actions conditioned on observations. We use these action samples to estimate the conditional action distribution and its entropy.

Given an observation o, which includes the task condition, we independently sample K initial noise inputs and generate action chunks of length H using the same policy, $\mathbf{A}^{(k)} = \mu_\theta(o, z^{(k)})$, where $z^{(k)} \sim \mathcal{N}(0, I)$. Here, μ_θ denotes the flow model's noise-to-action mapping, and $a_{h,d}^{(k)}$ is coordinate d of the action at position h in the k-th chunk. For continuous actions, we first compute the sample standard deviation in each coordinate and then estimate diagonal Gaussian differential entropy:

$$\begin{aligned} \widehat{\sigma}_{h,d} &= \mathrm{std}_k\left[a_{h,d}^{(k)}\right], \\ u_h^{\mathrm{cont}} &= \frac{1}{2}\sum_{d\in\mathcal{D}_{\mathrm{cont}}} \log\left[2\pi e\left(\widehat{\sigma}_{h,d}^2 + \lambda_{\mathrm{cov}}\right)\right]. \end{aligned} \tag{1}$$

Here, $\mathcal{D}_{\mathrm{cont}}$ is the set of continuous action coordinates, and $\lambda_{\mathrm{cov}} > 0$ ensures numerical stability. Appendix A.1 details the sample statistics, discrete gripper entropy, and the resulting action entropy u_h.

We observe that action entropy is low during most well-mastered stages and is typically high at a few decision-sensitive stages. As shown in Figure 2, we replace the base policy with a stronger policy at high- or low-entropy positions and compare task success rates at comparable replacement rates. Interventions at high-entropy positions yield larger gains in success rate, supporting the use of action entropy to identify actions with greater potential for policy improvement. This finding motivates entropy-based reweighting within each trajectory.

3.2. Uncertainty-Guided Training Reweighting

Existing post-training methods optimize policies using advantage estimates, Q-values, or supervised objectives. UACA uses action entropy to adjust the relative training weights of individual actions in these objectives.

Let b index action chunks in a training batch and $\mathcal{I}$ denote the set of valid action positions used for reweighting. We standardize action entropy and construct positive weights with unit mean:

$$\begin{aligned} \widetilde{u}_{b,h} &= \frac{u_{b,h} - \bar{u}}{\sigma_u + \epsilon}, \\ w_{b,h} &= (1-\eta) + \eta\,\frac{\exp(\widetilde{u}_{b,h}/\tau_w)}{|\mathcal{I}|^{-1}\sum_{(i,l)\in\mathcal{I}}\exp(\widetilde{u}_{i,l}/\tau_w)}. \end{aligned} \tag{2}$$

Here, $\bar{u}$ and σ_u are the mean and standard deviation of entropy over valid positions, while τ_w and η control the concentration of the weights and the reweighting strength, respectively. Higher entropy yields larger weights, which have unit mean and are treated as fixed coefficients during policy updates. Appendix A.1 defines the parameters.

3.3. Entropy-Guided Online Post-Training

As shown in Figure 3, UACA applies a common reweighting principle to the training signals of different post-training methods.

Advantage reweighting. For GRPO and PPO, we multiply the original advantages by the action entropy weights before updating the policy [7, 21, 33, 34]:

$$\widehat{A}^U_{b,h} = w_{b,h}\widehat{A}_{b,h}. \tag{3}$$

The sign of the advantage determines whether a sampled action is reinforced or suppressed, while the positive weight scales the corresponding update. The clipped objective is given in Appendix A.2.

Sustained high entropy in the tails of failed trajectories often accompanies unfamiliar states and repeated attempts that fail to restore task progress. We identify these segments and restore unit training weights, concentrating the additional learning effort on actions near and before the deviation. Appendix A.7 describes the failure-tail criterion.

Actor objective reweighting. In RL Token, DSRL, and EXPO-FT, critics guide the action policy, noise policy, and edit policy, respectively [10, 37, 40]. Let $Q_b(\phi)$ denote the value term for sample b in the original actor objective; the reweighted term is

$$\mathcal{L}^U_{\mathrm{value}}(\phi) = -\mathbb{E}_b[\bar{w}_b Q_b(\phi)]. \tag{4}$$

We obtain $\bar{w}_b$ by averaging the weights over executed action positions and normalizing them to unit mean, strengthening critic guidance for high-entropy action chunks. The actor objectives are detailed in Appendices A.3–A.5.

Supervised loss reweighting. We weight the supervision error at each action position in RL Token's action regression term, the flow-matching objective of online filtered behavior cloning (Filtered BC), and EXPO-FT's base-policy fine-tuning objective:

$$\mathcal{L}^{U}_{\text{sup}}(\theta) = \frac{1}{|\mathcal{I}|} \sum_{(b,h)\in\mathcal{I}} w_{b,h}\ell_{b,h}(\theta). \tag{5}$$

Here, $\ell_{b,h}$ denotes the original supervised loss at that position. The action regression loss fits reference actions, while the flow-matching loss fits conditional velocity targets; position-wise weights focus these objectives on actions for which the policy exhibits greater disagreement. The specific losses are given in Appendices A.3, A.5, and A.6.

These instantiations share the same action entropy estimator and reweighting mechanism and apply to a range of flow-based robot policies without training additional uncertainty models.

4. Conclusion

We propose Uncertainty-Aware Credit Assignment (UACA), which estimates policy uncertainty using action entropy to improve credit assignment in online post-training. UACA focuses learning on actions with greater potential for policy improvement without additional training, supports plug-and-play integration across flow-based robot policies and online post-training methods. Experiments across simulated and real-world environments, robot policies, and post-training methods demonstrate that UACA consistently improves sample efficiency and final policy performance.

References

[1] Philip J. Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *Proceedings of the 40th International Conference on Machine Learning*, pages 1577–1594. PMLR, 2023. 2

[2] Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, Haitian Liu, Ruowen Zhao, Yao Feng, Chendong Xiang, Yinze Rong, Hongyan Zhao, Hanyu Liu, Zhizhong Su, Lei Ma, Hang Su, and Jun Zhu. Motus: A unified latent action world model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 35101–35113, 2026. 1, 2

[3] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, brian ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40. PMLR, 2025. 1, 2

[4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, Laura Smith, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A Vision-Language-Action Flow Model for General Robot Control. In *Proceedings of Robotics: Science and Systems*, LosAngeles, CA, USA, 2025. 1, 2

[5] Jisong Cai, Long Ling, Shiwei Chu, Zhongshan Liu, Jiayue Kang, Zhixuan Liang, Wenjie Xu, Yinan Mao, Weinan Zhang, Xiaokang Yang, Ru Ying, Ran Zheng, and Yao Mu. Aha-wam:asynchronous horizon-adaptive world-action modeling with observation-guided context routing, 2026. 2

[6] Jialei Chen, Kai Wang, Kang Chen, Shuaihang Chen, Feng Gao, Wenhao Tang, Zhiyuan Li, Weilin Liu, Zhuyu Yao, Boxun Li, Yuanbo Xu, and Chao Yu. Lawam: Latent world action models for efficient dynamics-aware robot policies, 2026. 1, 2

[7] Kang Chen, Zhihao Liu, Tonghe Zhang, Zhen Guo, Si Xu, Hao Lin, Hongzhi Zang, Xiang Li, Quanlu Zhang, Zhaofei Yu, Guoliang Fan, Tiejun Huang, Yu Wang, and Chao Yu. π_{RL}: Online rl fine-tuning for flow-based vision-language-action models, 2026. 1, 2, 4

[8] Qianzhong Chen, Justin Yu, Mac Schwager, Pieter Abbeel, Fred Shentu, and Philipp Wu. Sarm: Stage-aware reward modeling for long horizon robot manipulation. In *International Conference on Learning Representations*, pages 17091–17116, 2026. 1, 3

[9] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44 (10-11):1684–1704, 2025. 2

[10] Perry Dong, Kuo-Han Hung, Tian Gao, Dorsa Sadigh, and Chelsea Finn. Expo-ft: Sample-efficient reinforcement learning finetuning for vision-language-action models, 2026. 1, 2, 4

[11] Perry Dong, Suvir Mirchandani, Dorsa Sadigh, and Chelsea Finn. What matters for batch online reinforcement learning in robotics? In *International Conference on Learning Representations*, pages 112294–112308, 2026. 2

[12] Tongyan Fang, Siyuan Huang, Naiyu Fang, Ganlong Zhao, Zhongjin Luo, Jianbo Liu, Xiaogang Wang, Ying Dong, and Hongsheng Li. Hierarchical advantage weighting for online rl fine-tuning of vlas from sparse episode outcomes, 2026. 1, 3

[13] Yu Fang, Wanxi Dong, Jiaqi Liu, Yue Yang, Mingxiao Huo, Yao Mu, Huaxiu Yao, Li Erran Li, Daniel Szafir, and Mingyu Ding. Densereward: Dense reward learning via failure synthesis for robotic manipulation, 2026. 1

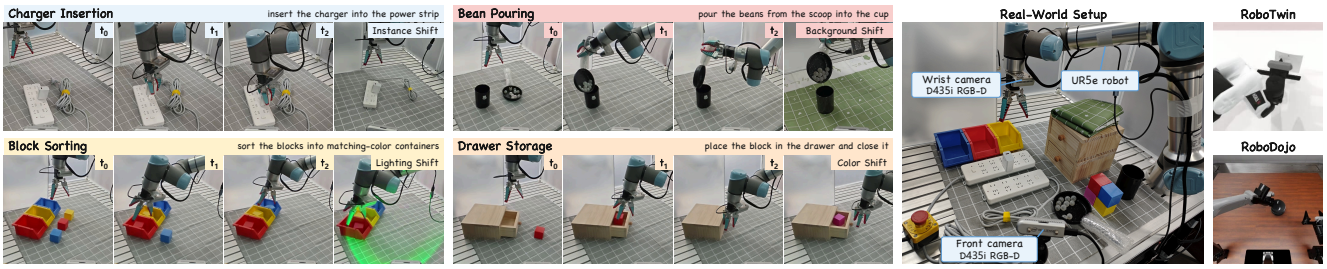


Figure 4. The main experimental settings include RoboTwin, RoboDojo, and four real-world tasks on the UR5e platform. Image sequences show successful task execution; the final image for each task illustrates a form of domain randomization.

[14] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training. In *Advances in Neural Information Processing Systems*, pages 46375–46408. Curran Associates, Inc., 2025. 3

[15] Xiangdong Feng, Yuxuan Cheng, Chen Shi, Boyao Han, Yuxuan Yan, Yitong Hong, Zhuotao Tian, and Li Jiang. Denoising tells when to replan: Denoising-variance adaptive chunking for flow-based robot policies, 2026. 3

[16] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870. PMLR, 2018. 1, 2

[17] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 24328–24346. PMLR, 2025. 2

[18] Dongchi Huang, Hongyin Zhang, Bohan Hou, Siteng Huang, Zhian Su, Hang Guo, Tong Lu, Zhaofeng Xu, Jiahao Tang, Jianfei Yang, Donglin Wang, Peixi Peng, Mingxiu Chen, Deli Zhao, and Xin Li. Rynnvalue: Scaling robotic value foundation models with temporal distance, 2026. 1, 3

[19] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos policy: Fine-tuning video models for visuomotor control and planning. In *International Conference on Learning Representations*, pages 71531–71552, 2026. 1, 2

[20] Thanh-Long V. Le, Myeongho Jeon, Kim Vu, Viet Lai, and Eunho Yang. No prompt left behind: Exploiting zero-variance prompts in llm reinforcement learning via entropy-guided advantage shaping. In *International Conference on Learning Representations*, pages 121956–121982, 2026. 3, 1

[21] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Yang Zhaohui, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, Dehui Wang, Dingxiang Luo, Yuchen Fan, Youbang Sun, Jia Zeng, Jiangmiao Pang, Shanghang Zhang, Yu Wang, Yao Mu, Bowen Zhou, and Ning Ding. Simplevla-rl: Scaling vla training via reinforcement learning. In *International Conference on Learning Representations*, pages 125846–125869, 2026. 1, 2, 4

[22] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Luyao Zhang, Mingrui Yu, Zelin Gao, Nan Xue, Boyu Zhou, Xing Zhu, Mingyu Ding, Yujun Shen, and Yinghao Xu. Causal World Modeling for Robot Control. In *Proceedings of Robotics: Science and Systems*, Sydney, Australia, 2026. 2

[23] Yuyang Liu, Chuan Wen, Yihang Hu, Dinesh Jayaraman, and Yang Gao. Timerewarder: Learning dense reward from passive videos via frame-wise temporal distance. In *Forty-third International Conference on Machine Learning*, 2026. 1

[24] Guanxing Lu, Rui Zhao, Haitao Lin, He Zhang, and Yansong Tang. Human-in-the-loop online rejection sampling for robotic manipulation, 2025. 2

[25] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105): eads5033, 2025. 1, 2

[26] Ashvin Nair, Abhishek Gupta, Murtaza Dalal, and Sergey Levine. Awac: Accelerating online reinforcement learning with offline datasets, 2021. 1, 3

[27] Seohong Park, Qiyang Li, and Sergey Levine. Flow q-learning. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 48104–48127. PMLR, 2025. 2

[28] Daojie Peng, Fulong Ma, Jiahang Cao, Qiang Zhang, Xupeng Xie, Jian Guo, Ping Luo, Andrew F. Luo, Boyu Zhou, and Jun Ma. Attena+: Rectifying action inequality in robotic foundation models, 2026. 3

[29] Ziyang Rao, Yiren Zhao, Weiyu Guo, Ben Fei, Yandong Guo, and Hui Xiong. The geometry of flow-matching uncertainty: A cost-free uncertainty proxy and its application in flow-based vla failure detection, 2026. 3

[30] Haoxiang Ren, Yifei Sheng, Zhilong Zhang, Runjie Xu, Bowen Lu, Haoxin Lin, and Yang Yu. When to trust your teacher: Uncertainty-aware on-policy distillation for embodied policies. In *RSS 2026 Workshop on Diffusion for Robot Learning*, 2026.

[31] Ralf Römer, Adrian Kobras, Luca Worbis, and Angela Schoellig. Failure prediction at runtime for generative robot policies. In *Advances in Neural Information Processing Systems*, pages 7631–7670. Curran Associates, Inc., 2025. 3

[32] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous

control using generalized advantage estimation. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. 1

[33] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. 4, 1

[34] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. 4, 1

[35] Yifei Sheng, Haoxiang Ren, Zhilong Zhang, Haonan Wang, Runjie Xu, Yihao Sun, Nan Tang, Zhichao Wu, Lei Yuan, Haoxin Lin, and Yang Yu. Prioritized rollouts for efficient world model-based vision-language-action policy optimization, 2026. 3

[36] Huajie Tan, Sixiang Chen, Yijie Xu, Zixiao Wang, Cheng Chi, Yuheng Ji, Yaoxu Lyu, Zhongxia Zhao, Xiansheng Chen, Peterson Co, Shaoxuan Xie, Guocai Yao, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. General process reward modeling for robotic reinforcement learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22412–22422, 2026. 1

[37] Andrew Wagenmaker, Yunchu Zhang, Mitsuhiko Nakamoto, Seohong Park, Waleed Yagoub, Anusha Nagabandi, Abhishek Gupta, and Sergey Levine. Steering your diffusion policy with latent space reinforcement learning. In *Proceedings of The 9th Conference on Robot Learning*, pages 258–282. PMLR, 2025. 1, 2, 4

[38] Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xiong-Hui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning. In *Advances in Neural Information Processing Systems*, pages 115452–115486. Curran Associates, Inc., 2025. 3

[39] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. Openwam: An open, modular exploration towards systematic world-action model pretraining, 2026. 1, 2

[40] Charles Xu, Jost Tobias Springenberg, Michael Equi, Ali Amin, Adnan Esmail, Sergey Levine, and Liyiming Ke. Rl token: Bootstrapping online rl with vision-language-action models, 2026. 1, 2, 4

[41] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. World action models are zero-shot policies, 2026. 2

[42] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-wam: Do world action models need test-time future imagination?, 2026. 1, 2

[43] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024. 2

[44] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Tai Wang, Ya-Qin Zhang, Jingjing Liu, and Xianyuan Zhan. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations*, pages 60580–60606, 2026. 2

When Policies Hesitate: Uncertainty-Aware Credit Assignment for Flow-Based Robot Policy Optimization

Supplementary Material

A. Method Details and Instantiations

A.1. Action Entropy Estimation and Weight Construction

Action entropy estimation. We use the model's normalized coordinates for continuous actions. Given K action samples conditioned on the same observation, we compute the sample mean and standard deviation at each action position and coordinate:

$$\bar{a}_{h,d} = \frac{1}{K}\sum_{k=1}^{K} a_{h,d}^{(k)}, \qquad \widehat{\sigma}_{h,d} = \sqrt{\frac{1}{K-1}\sum_{k=1}^{K}\left(a_{h,d}^{(k)} - \bar{a}_{h,d}\right)^2}. \tag{6}$$

Substituting the standard deviation into Equation (1) gives diagonal Gaussian differential entropy. All logarithms are natural.

For discrete gripper states, we compute entropy from the empirical frequencies $\widehat{p}_h(g)$:

$$u_h^{\text{grip}} = -\sum_{g\in\mathcal{G}} \widehat{p}_h(g)\log \widehat{p}_h(g). \tag{7}$$

We use the convention $0\log 0 = 0$. The entropy at each position is $u_h = u_h^{\text{cont}} + \kappa u_h^{\text{grip}}$, where κ controls the contribution of the gripper component; continuous gripper coordinates are included among the continuous actions.

Weight normalization. Equation (2) computes the mean and standard deviation of entropy over valid action positions, using the population standard deviation within this set. The temperature $\tau_w > 0$ controls weight concentration, $\eta \in [0,1]$ controls reweighting strength, and $\epsilon > 0$ stabilizes standardization. When $\eta = 0$, all weights equal 1. The weights are treated as fixed coefficients during updates [20].

When the Q-function evaluates an executed action chunk, we average the weights over its executed prefix and normalize them to unit mean:

$$\omega_b = \frac{1}{C_b}\sum_{h=1}^{C_b} w_{b,h}, \qquad \bar{w}_b = \frac{\omega_b}{B^{-1}\sum_{i=1}^{B}\omega_i}. \tag{8}$$

Here, C_b is the execution horizon and B is the number of action chunks in the batch. Per-action supervision uses $w_{b,h}$, while the value term uses $\bar{w}_b$.

A.2. Advantage Reweighting for GRPO and PPO

GRPO computes advantages from relative returns within a group of trajectories; under outcome supervision, all actions in a trajectory share the same advantage [21, 34]:

$$\widehat{A}_i = \frac{R_i - \bar{R}}{\sigma_R + \epsilon}. \tag{9}$$

Here, R_i is the trajectory return, and $\bar{R}$ and σ_R are the group mean and standard deviation. PPO computes advantages from value estimates, for example through generalized advantage estimation [7, 32, 33].

UACA multiplies the original advantages by action entropy weights in the clipped objective:

$$\mathcal{J}_U(\theta) = \mathbb{E}_{b,h}\Big[\min\Big\{\rho_{b,h}(\theta)\widehat{A}^U_{b,h}, \; \text{clip}\big(\rho_{b,h}(\theta), 1-\varepsilon_{\text{clip}}, 1+\varepsilon_{\text{clip}}\big)\,\widehat{A}^U_{b,h}\Big\}\Big]. \tag{10}$$

Here, $\rho_{b,h}$ is the policy ratio used by the baseline at that position, and $\varepsilon_{\text{clip}}$ is the clipping threshold. For flow-based policies, we use the policy ratio formulation of π_{RL} [7]. When advantages are shared within a trajectory, $\widehat{A}_{b,h}$ takes the advantage of the corresponding trajectory.

Positive weights preserve the sign of the advantage while adjusting each action's relative contribution to the policy update. Advantages are estimated and normalized before entropy weighting, and final weights for failure tails follow Appendix A.7.

A.3. Value and Action Regression Terms in RL Token

After representation adaptation, RL Token freezes the VLA and learns lightweight actor and critic networks online. The actor generates executed actions conditioned on the RL token, proprioceptive state, and VLA reference actions; its objective combines value maximization with regression toward the reference actions [40].

Let x_b denote the input representation, $\widetilde{\mathbf{A}}_b$ the VLA reference actions, and $\mathbf{A}_b$ the executed action chunk generated by the actor. The UACA objective is

$$\mathcal{L}^U_{\text{RLT}}(\phi) = \mathbb{E}_b\Big[-\bar{w}_b Q_\psi(x_b, \mathbf{A}_b) + \beta\sum_{h=1}^{C_b} w_{b,h}\|a_{b,h} - \widetilde{a}_{b,h}\|_2^2\Big]. \tag{11}$$

Here, β controls the strength of action regression, and C_b is the execution horizon. Weights are computed from action samples generated by the reference VLA with different initial noise samples. We weight the value term at the chunk level and action regression at the position level; the former strengthens critic guidance at observations with high action entropy, and the latter increases the emphasis on fitting the corresponding actions.

A.4. Noise Policy Optimization in DSRL

DSRL freezes the pretrained generative policy and learns a policy over initial noise, $\pi_\phi^z(z \mid o)$, which changes the generated actions through $\mathbf{A} = \mu_\theta(o, z)$. Its noise-aliased instantiation trains a noise critic $Q_\omega^{\mathcal{W}}(o, z)$ using value estimates from an action critic, enabling behavior optimization in noise space [37].

For SAC-based instantiations, UACA applies the action-chunk weight to the actor's Q-term [16]:

$$\mathcal{L}^U_{\mathrm{DSRL}}(\phi) = \mathbb{E}_{o\sim\mathcal{D},\, z\sim\pi_\phi^z}\Big[\alpha \log \pi_\phi^z(z \mid o) - \bar{w}(o) Q_\omega^{\mathcal{W}}(o, z)\Big]. \tag{12}$$

Weights are computed from action samples generated by the fixed base policy with standard Gaussian initial noise. The SAC entropy coefficient α regularizes the noise policy, whereas UACA weights are derived from the entropy of generated actions. Greater action disagreement strengthens critic guidance at the corresponding observations, focusing noise-policy learning on improving behavior in these states.

A.5. Edit Policy and Base-Policy Fine-Tuning in EXPO-FT

EXPO-FT uses a lightweight edit policy to modify the base VLA's executed action chunks and further fine-tunes the base policy with online experience [10]. Let $\mathbf{A}_{1:C}$ denote the executed prefix of the base action chunk and $\Delta\mathbf{A}$ the edit; the candidate action chunk is $\mathbf{A}^{\mathrm{edit}} = \mathbf{A}_{1:C} + \Delta\mathbf{A}$. UACA weights the edit policy's Q-term:

$$\mathcal{L}^U_{\mathrm{edit}}(\phi) = \mathbb{E}_b\Big[\alpha \log \pi_\phi^{\mathrm{edit}}(\Delta\mathbf{A}_b \mid o_b, \mathbf{A}_{b,1:C}) - \bar{w}_b Q_\psi(o_b, \mathbf{A}_b^{\mathrm{edit}})\Big]. \tag{13}$$

The base policy retains its original supervised action targets. For a flow-based policy, let $\ell_{b,h}^{\mathrm{FM}}$ denote its per-action flow-matching loss; UACA uses

$$\mathcal{L}^U_{\mathrm{base}}(\theta) = \mathbb{E}\left[\frac{1}{|\mathcal{I}|}\sum_{(b,h)\in\mathcal{I}} w_{b,h}\ell_{b,h}^{\mathrm{FM}}(\theta)\right]. \tag{14}$$

Appendix A.6 gives the flow-matching loss. Both weights are derived from the base VLA's action entropy: chunk-level weights strengthen critic guidance for the edit policy, while position-wise weights focus base-policy fitting on high-entropy actions.

A.6. Online Filtered Behavior Cloning

Online filtered behavior cloning (Filtered BC) alternates between trajectory collection and policy updates, using successful experience for further training [11, 24]. Let $\mathcal{D}^+$ denote the filtered training set, with action targets $\mathbf{A}^*$.

Using the flow-time convention with actions at 0 and noise at 1, we set $x_t = (1-t)\mathbf{A}^* + tz$, where $z \sim \mathcal{N}(0, I)$ and t follows the base policy's training distribution. The weighted flow-matching objective is

$$\mathcal{L}^U_{\mathrm{FBC}}(\theta) = \mathbb{E}_{(o,\mathbf{A}^*)\sim\mathcal{D}^+,\, t,z}\Big[\frac{1}{H}\sum_{h=1}^{H} w_h(o) \cdot \|v_\theta(x_t, t, o)_h - (z_h - a_h^*)\|_2^2\Big]. \tag{15}$$

Here, v_θ is the velocity field, and the squared error fits the conditional velocity target [27]. Trajectory filtering selects successful experience, while action entropy weights allocate fitting effort within that experience. Base-policy fine-tuning in Appendix A.5 uses the same per-action loss.

A.7. Restoring Unit Weights in Failure Tails

After a failure, a trajectory may enter unfamiliar states and exhibit sustained high action entropy during subsequent attempts. We identify these segments using execution outcomes and persistent high entropy, then restore unit training weights.

Let $u_{1:T}$ denote the entropy sequence of executed actions. For a failed trajectory, t^* is the start of the first run of L consecutive actions whose entropy is at least q, where q is an entropy quantile estimated from a calibration set. Actions up to and including this point retain their entropy weights, while subsequent actions receive unit weights:

$$w_t^{\mathrm{tail}} = \begin{cases} 1, & \text{failed trajectory and } t > t^*, \\ w_t, & \text{otherwise.} \end{cases} \tag{16}$$

We set $t^* = T$ for successful trajectories and trajectories without a sustained high-entropy segment. Entropy weights are normalized over the retained positions using Equation (2); if the entire batch consists of failure-tail positions, all weights are set to 1. The resulting w_t^{tail} is used for advantage reweighting, concentrating additional learning effort on actions near and before the deviation. L is measured in executed actions.